

Podstawy Javy

Typy proste, obiekty, referencje na obiekty - analiza porównawcza Javy z C++

- źródło: [GeeksforGeeks - Primitive Data Type vs Object Data Type in Java](#)
Typy proste (int, char, boolean) przechowują bezpośrednio wartości i są alokowane na stosie. Typy obiektowe (referencyjne) przechowują adres pamięci (referencję) wskazujący na właściwy obiekt zaalokowany na stercie. W przeciwieństwie do C++, w Javie brak jest wskaźników operujących bezpośrednio na pamięci (brak arytmetyki wskaźników), a zarządzanie alokacją i zwalnianiem pamięci odbywa się automatycznie przez Garbage Collector. Referencja w Javie przypomina wskaźnik z C++, ale jest bezpieczna i pozbawiona możliwości bezpośredniej manipulacji adresem.

Czym jest klonowanie i kopiowanie obiektu?

- źródła: [GeeksforGeeks - Cloneable Interface](#) | [Baeldung - Java Copy Constructor](#)
Klonowanie to proces tworzenia dokładnej kopii istniejącego obiektu za pomocą metody clone() z klasy Object. Klasa klonowana musi implementować interfejs znacznikowy Cloneable (inaczej zgłaszany jest CloneNotSupportedException). Klonowanie płytkie (*shallow copy*) kopiuje jedynie wartości typów prostych i referencje do obiektów, co sprawia, że oryginalny i sklonowany obiekt dzielą te same podobiekty. Kopiowanie głębokie (*deep copy*) tworzy nowe kopie wszystkich powiązanych obiektów. Alternatywą jest konstruktor kopiujący (*copy constructor*), który przyjmuje jako parametr obiekt tej samej klasy i ręcznie przepisuje jego pola.

Składnia i działanie konstruktora

- źródła: [Oracle - Constructors](#) | [Passing Arguments](#) | [This Keyword](#) | [Super Keyword](#)
Konstruktor to specjalny blok kodu wywoływany podczas tworzenia instancji klasy operatorem new. Nie posiada zwracanego typu, a jego nazwa jest identyczna z nazwą klasy.
- Wywołanie konstruktora tej samej klasy odbywa się za pomocą słowa kluczowego this(...) i musi być pierwszą instrukcją w konstruktorze.
- Wywołanie konstruktora klasy nadrzędnej realizowane jest poprzez super(...) i również musi stanowić pierwszą linię kodu.

```
Java
class Parent {
    Parent(String msg) { System.out.println(msg); }
}

class Child extends Parent {
    Child() {
        this("Domyślny parametr"); // Wywołanie konstruktora tej samej klasy
    }
}
```

```
Child(String arg) {
    super("Konstruktor rodzica: " + arg); // Wywołanie konstruktora klasy nadrzędnej
}
}
```

Sposoby użycia słowa kluczowego final

- źródło: [Baeldung - Java Final Keyword](#)
Słowo kluczowe final służy do blokowania możliwości modyfikacji/rozszerzania:
- 1. **Zmienne:** Tworzy stałą (zmienna primitive/referencyjna nie może zmienić swojej wartości lub adresu po przypisaniu).
- 2. **Metody:** Zabezpiecza metodę przed nadpisaniem (*override*) w klasach potomnych.
- 3. **Klasy:** Uniemożliwia dziedziczenie po danej klasie (np. klasa String).

Programowanie obiektowe

Dziedziczenie klas a implementacja interfejsów

- źródło: [GeeksforGeeks - extends vs implements](#)
Dziedziczenie klas realizowane jest słowem kluczowym extends. Klasa może dziedziczyć tylko po jednej klasie bazowej (brak wielodziedziczenia klas w Javie). Implementacja interfejsów odbywa się za pomocą implements – jedna klasa może implementować wiele interfejsów naraz. Dziedziczenie klas tworzy relację "jest czymś" (*is-a*), natomiast interfejs definiuje kontrakt i zestaw zachowań, które klasa zobowiązuje się dostarczyć.

Klasy abstrakcyjne a interfejsy

- źródło: [Baeldung - Java Interface vs Abstract Class](#)
Klasa abstrakcyjna może posiadać stan (zwykle pola), stanowy stan prywatny, konstruktory oraz metody z pełną implementacją i abstrakcyjne. Interfejsy służą głównie do definiowania kontraktów. Od Javy 8 interfejsy mogą zawierać metody domyślne (default) oraz statyczne, a od Javy 9 także metody prywatne. Klasa abstrakcyjna używana jest do współdzielenia kodu w hierarchii spójnych klas, zaś interfejs łączy niepowiązane klasy wspólnym zachowaniem.

Polimorfizm

- źródło: [Baeldung - Polymorphism in Java](#)
Polimorfizm (wielopostaciowość) to zdolność obiektów do reagowania w różny sposób na to samo wywołanie metody.
- **Polimorfizm statyczny (w czasie kompilacji):** Przeciążanie metod (*overloading*) – ta sama nazwa metody, ale różna liczba lub typy parametrów wewnątrz tej samej klasy.
- **Polimorfizm dynamiczny (w czasie wykonywania):** Nadpisywanie metod (*overriding*) – klasa pochodna dostarcza własną implementację metody zdefiniowanej w klasie bazowej lub interfejsie. Decyzja o tym, która metoda zostanie wywołana, zapada dynamicznie w trakcie działania programu na podstawie rzeczywistego typu obiektu.

Enkapsulacja i modyfikatory dostępu

- źródło: [GeeksforGeeks - Access Modifiers in Java](#)
Enkapsulacja (hermetyzacja) polega na ukrywaniu wewnętrznego stanu obiektu i udostępnianiu go jedynie poprzez kontrolowane metody (getter i setter). Modyfikatory

dostępu sterują widocznością:

Modyfikator	Ta sama klasa	Ten sam pakiet	Klasa potomna	Świat (Wszędzie)
private	Tak	Nie	Nie	Nie
default (brak)	Tak	Tak	Nie	Nie
protected	Tak	Tak	Tak	Nie
public	Tak	Tak	Tak	Tak

Zagnieżdżanie klas: Klasy wewnętrzne (zestawione wewnątrz innej klasy) mają dostęp do wszystkich pól i metod klasy otaczającej, w tym prywatnych (private).

Rodzaje asocjacji między klasami

- źródło: Baeldung - Association, Aggregation, and Composition in Java
- **Asocjacja:** Ogólna relacja między dwoma osobnymi klasami bez narzucania zależności cyklu życia (np. *Nauczyciel – Uczeń*).
- **Agregacja:** Relacja typu "has-a" ze słabym powiązaniem. Obiekt zależny może istnieć niezależnie od obiektu nadrzędnego (np. *Dział firmowy – Pracownik*; po usunięciu działu pracownik nadal istnieje).
- **Kompozycja:** Ścisła relacja "has-a". Cykl życia obiektu podrzędnego jest całkowicie zależny od obiektu nadrzędnego (np. *Dom – Pokój*; usunięcie domu niszczy pokój).

[Asocjacja] KlasaA -----> KlasaB
[Agregacja] Samochód <>----- Kolo
[Kompozycja] Budynek <■>----- Pokój

Wzorce projektowe

Singleton

- źródło: Baeldung - Singletons in Java
Gwarantuje istnienie tylko jednej instancji danej klasy w aplikacji oraz zapewnia do niej globalny punkt dostępu.
- **Konstrukcja:** Prywatny konstruktor, prywatne pole statyczne przechowujące instancję oraz publiczna statyczna metoda dostępowa (np. `getInstance()`).
- **Zastosowanie:** Klasy konfiguracji, pule połączeń z bazą danych, loggery.
- **Ograniczenia/Problemy:** Trudności w testowaniu (trudno zamienić na Mocki), złamanie

zasady pojedynczej odpowiedzialności (SRP), problemy z wielowątkowością (wymaga synchronizacji/podwójnego sprawdzania locków lub użycia klas statycznych/enumów), podatność na złamanie przez refleksję lub deserializację.

Budowniczy (Builder)

- źródło: [Refactoring Guru - Builder](#)

Rozdziela proces tworzenia złożonego obiektu od jego reprezentacji, pozwalając na tworzenie różnych wariantów obiektu przy użyciu tego samego kodu konstrukcyjnego. Używany do uniknięcia konstruktorów z długą listą parametrów ("telescoping constructor"). Pozwala na stopniowe ustawianie pól obiektu za pomocą łańcuchowania metod (fluent interface) oraz zwieńczenie procesu metodą build().

Dekorator (Decorator)

- źródło: [Refactoring Guru - Decorator](#)

Strukturalny wzorzec projektowy pozwalający na dynamiczne dodawanie nowych obowiązków i zachowań obiektom poprzez umieszczanie ich w specjalnych obiektach-opakowaniach (wrapperach). Dekorator implementuje ten sam interfejs co obiekt opakowywany i deleguje do niego pracę, dokładając własną funkcjonalność przed lub po wywołaniu metody (np. dodanie szyfrowania/kompresji do strumienia danych w java.io).

Kolekcje

Interfejs Collection

- źródło: [Oracle JavaDocs - Collection](#)

Główny interfejs w hierarchii kolekcji Javy (znajdujący się na szczycie obok interfejsu Map, który nie dziedziczy po Collection). Definiuje podstawowe operacje wykonywane na grupach obiektów: dodawanie (add), usuwanie (remove), sprawdzanie rozmiaru (size), czyszczenie (clear), sprawdzanie obecności (contains) oraz przekształcanie w tablicę (toArray).

Porównanie LinkedList i ArrayList

- źródło: [Baeldung - ArrayList vs LinkedList in Java](#)

Cecha	ArrayList	LinkedList
Struktura	Dynamiczna tablica	Dwukierunkowa lista wiązana
Dostęp po indeksie get(i)	Bardzo szybki $O(1)$	Wolny $O(n)$
Wstawianie/Usuwanie na końcu	Zamortyzowane $O(1)$	Bardzo szybkie $O(1)$
Wstawianie/Usuwanie w środku	Wolne $O(n)$ – wymaga przesunięcia elementów	Szybkie po dotarciu do węzła $O(1)$

Narzut pamięciowy	Mały	Duży (dodatkowe wskaźniki na węzłach)
-------------------	------	---------------------------------------

Rekonstrukcja działania std::multimap z C++ w Javie

- źródła: [cppreference - std::multimap](#) | [Oracle JavaDocs - Map](#)
W C++ std::multimap pozwala na przechowywanie wielu wartości pod tym samym kluczem. W standardowej bibliotece Javy brak bezpośredniego odpowiednika w interfejsie Map. Działanie to rekonstruuje się poprzez stworzenie mapy zawierającej kolekcję jako wartość: Map<K, List<V>> lub Map<K, Set<V>>. Dodanie elementu wymaga sprawdzenia czy klucz istnieje – jeśli nie, tworzy się nową listę/zbiór, a następnie dodaje wartość.

Pliki

Readery - podział, różnice, zastosowanie

- źródła: [GeeksforGeeks - Reader Class](#) | [BufferedReader](#) | [InputStreamReader](#)
Reader to abstrakcyjna klasa bazowa dla odczytu strumieni znakowych (16-bitowe znaki Unicode).
- **InputStreamReader**: Działa jako most ze strumieni bajtowych do znakowych. Odczytuje bajty i dekoduje je na znaki przy użyciu określonego zestawu kodowania (np. UTF-8).
- **BufferedReader**: Opakowuje inny Reader i buforuje odczytywane znaki w pamięci podręcznej, co znacząco podnosi wydajność odczytu. Udostępnia wygodną metodę readLine() do odczytu tekstu linia po linii.

Serializacja obiektów. Zapis i odczyt obiektów z pliku

- źródło: [Baeldung - Serialization in Java](#)
Serializacja to proces przekształcania stanu obiektu do postaci strumienia bajtów, co umożliwia jego zapis na dysku lub przesłanie siecią. Klasa obiektu musi implementować interfejs znacznikowy Serializable. Pola oznaczone modyfikatorem transient są pomijane przy serializacji. Zapis obiektu wykonuje się przy użyciu klasy ObjectOutputStream, a odczyt za pomocą ObjectInputStream i metody readObject().

Wyjątki

Składnia i zastosowanie wyjątków

- źródło: [GeeksforGeeks - Exceptions in Java](#)
Wyjątki służą do obsługi błędów powstających podczas działania programu. Składnia opiera się na blokach try (kod potencjalnie niebezpieczny), catch (przechwycenie i obsługa wyjątku) oraz opcjonalnym finally (kod wykonywany zawsze, np. zwalnianie zasobów). Wyjątek wyrzuca się słowem throw, a w sygnaturze metody deklaruje słowem throws.

Exception a RuntimeException - różnica i przykłady

- źródła: [Oracle JavaDocs - Exception](#) | [RuntimeException](#)
- **Wyjątki kontrolowane (Checked Exceptions)**: Dziedziczą po Exception (ale nie po RuntimeException). Muszą być jawnie obsługiwane w bloku try-catch lub zadeklarowane w sygnaturze metody za pomocą throws. Przykład: IOException, SQLException.
- **Wyjątki niekontrolowane (Unchecked Exceptions)**: Dziedziczą po RuntimeException.

Reprezentują błędy w logice programu; kompilator nie wymaga ich jawnej obsługi.
Przykład: `NullPointerException`, `ArrayIndexOutOfBoundsException`.

Wyrażenie `try-with-resources`

- źródło: [Oracle Java Tutorial - The try-with-resources Statement](#)
Składnia wprowadzona w Javie 7, która automatycznie zamyka zasoby otwarte w deklaracji `try(...)`. Dowolny obiekt, który implementuje interfejs `java.lang.AutoCloseable` lub `java.io.Closeable`, może zostać użyty w tej konstrukcji. Zapewnia to bezpieczne zwalnianie zasobów bez konieczności pisania bloków `finally`.

```
Java
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
} // br zostanie automatycznie zamknięty w tym miejscu
```

Strumienie

Trzy metody pośrednie w strumieniu

- źródło: [GeeksforGeeks - Intermediate Methods of Stream in Java](#)
Metody pośrednie (*intermediate operations*) transformują strumień w inny strumień i są leniwie ewaluowane (wykonują się dopiero po wywołaniu metody terminalnej):
1. **`filter(Predicate)`**: Odfiltruje elementy na podstawie zadanego warunku.
 2. **`map(Function)`**: Przekształca każdy element strumienia poprzez zastosowanie do niego podanej funkcji.
 3. **`sorted()`**: Sortuje elementy strumienia według porządku naturalnego lub podanego komparatora.

Kolektory

- źródło: [Baeldung - Guide to Java 8 Collectors](#)
Kolektory (dostarczane przez klasę `Collectors`) są używane jako argumenty metody terminalnej `collect()`. Służą do agregowania i przekształcania elementów strumienia w końcowe struktury danych lub wartości. Przykładowe użycia:
- `Collectors.toList()` / `toSet()` – gromadzenie danych w liście lub zbiorze.
- `Collectors.joining(",")` – łączenie napisów z użyciem separatora.
- `Collectors.groupingBy(...)` – grupowanie elementów według zadanej kluczowej funkcji (tworzenie mapy).

Testowanie

Motywacja do testowania i Test Driven Development (TDD)

- źródło: [Baeldung - Unit Testing vs TDD](#)
Testy jednostkowe zapewniają poprawność działania poszczególnych komponentów,

redukują ryzyko regresji i ułatwiają refaktoryzację kodu. TDD (Test-Driven Development) to technika wytwarzania oprogramowania oparta na cyklu:

1. **Red:** Napisz prosty test jednostkowy opisujący nową funkcjonalność, który nie przechodzi.
2. **Green:** Napisz minimalną ilość kodu produkcyjnego wystarczającą do zaliczenia testu.
3. **Refactor:** Zoptymalizuj i oczyść kod produkcyjny i testowy, upewniając się, że testy nadal przechodzą.

Parametryzacja testów

- źródło: [Baeldung - Parameterized Tests in JUnit 5](#)
Pozwala na wielokrotne uruchamianie tego samego testu z różnymi zestawami danych wejściowych bez konieczności powielania kodu testowego. W JUnit 5 wykorzystuje się do tego adnotację `@ParameterizedTest` połączoną ze źródłem danych, takim jak: `@ValueSource` (prosta tablica wartości), `@CsvSource` (wiersze danych w formacie CSV) lub `@MethodSource` (metoda zwracająca strumień/listę argumentów).

Pakiety

Zasady tworzenia i zastosowanie pakietów

- źródło: [Oracle Java Tutorial - Creating and Using Packages](#)
Pakiety służą do grupowania powiązanych ze sobą klas i interfejsów, zapobiegania konfliktom nazw (przestrzenie nazw) oraz kontrolowania dostępu. Konwencja nazewnictwa zaleca używanie odwróconych nazw domen internetowych w małych literach (np. `com.example.project`). Deklaracja pakietu `package com.example;` musi znajdować się na samej górze pliku źródłowego.

Pliki JAR, ich tworzenie, wykorzystanie i manifest

- źródło: [Oracle Java Tutorial - Packaging Programs in JAR Files](#)
Plik JAR (Java Archive) to format archiwum oparty na formacie ZIP, służący do pakowania wielu skompilowanych plików `.class`, zasobów oraz metadanych w pojedynczy plik. Tworzy się go narzędziem `jar` z wiersza poleceń (np. `jar cf app.jar *.class`). Wewnątrz archiwum w katalogu `META-INF/` znajduje się plik `MANIFEST.MF`. Manifest przechowuje metadane – np. wpis `Main-Class: com.example.Main` definiuje punkt wejścia do aplikacji wywoływanej komendą `java -jar app.jar`.

Trzy zastosowania Mavena

- źródło: [Baeldung - Apache Maven Tutorial](#)
- 1. **Zarządzanie zależnościami:** Automatyczne pobieranie i dołączanie bibliotek zewnętrznych wraz z ich zależnościami przechodnimi z centralnych repozytoriów.
- 2. **Automatyzacja procesu budowania:** Standaryzacja cyklu życia projektu (kompilacja, uruchamianie testów, pakowanie do JAR/WAR).
- 3. **Zarządzanie strukturą i spójnością projektu:** Wymuszenie jednolitej struktury katalogów oraz łatwe zarządzanie wersjami oprogramowania i wtyczkami.

Elementy składowe pliku `pom.xml`

- źródło: [Apache Maven - Introduction to the POM](#)
Plik `pom.xml` (Project Object Model) stanowi serce konfiguracji Mavena. Główne sekcje:
- **GAV (Coordinates):** `groupId`, `artifactId`, `version` – unikalny identyfikator projektu.
- `<dependencies>`: Lista zewnętrznych bibliotek wymaganych przez projekt.

- <plugins>: Narzędzia rozszerzające proces budowania aplikacji.
- <properties>: Zmienne globalne (np. wersja kompilatora Javy).

Programowanie generyczne

Typy generyczne a typ Object

- źródło: [Oracle Java Tutorial - Generic Types](#)
Przed wprowadzeniem typów generycznych w Javie 5 używano typu Object, co wymagało rzutowania typów przy odczycie i narażało aplikację na błędy w czasie wykonywania (ClassCastException). Typy generyczne umożliwiają parametryzowanie klas i metod typami. Przenoszą sprawdzanie poprawności typów na etap kompilacji oraz eliminują potrzebę jawnego rzutowania. Mechanizm wymazywania typów (*type erasure*) usuwa informacje o typach generycznych podczas kompilacji na potrzeby kompatybilności wstecznej.

Ograniczenia typów generycznych

- źródło: [Oracle Java Tutorial - Wildcards](#)
Do ograniczania typów w generykach używa się symboli wieloznacznych (wildcard - ?):
- **Upper Bounded Wildcards (? extends T)**: Ogranicza typ do klasy T lub jej klas potomnych. Używane, gdy dane są odczytywane ze struktury (zasada PECS: *Producer Extends*).
- **Lower Bounded Wildcards (? super T)**: Ogranicza typ do klasy T lub jej nadklas. Używane, gdy dane są zapisywane do struktury (*Consumer Super*).
- **Unbounded Wildcards (?)**: Typ niezdefiniowany, używany gdy operacje nie zależą od typu parametru.

Interfejsy funkcyjne: Supplier, Predicate, Consumer, Function

- źródło: [Oracle JavaDocs - java.util.function](#)
- **Supplier<T>**: Nie przyjmuje argumentów, zwraca wartość typu T (metoda T get()).
- **Predicate<T>**: Przyjmuje argument typu T, zwraca boolean (metoda boolean test(T t)).
- **Consumer<T>**: Przyjmuje argument typu T, nie zwraca nic (metoda void accept(T t)).
- **Function<T, R>**: Przyjmuje argument typu T, zwraca wynik typu R (metoda R apply(T t)).

Interfejsy funkcyjne a funkcje lambda, własna implementacja

- źródła: [JavaTpoint - Functional Interfaces](#) | [Oracle Java Tutorial - Lambda Expressions](#)
Interfejs funkcyjny posiada dokładnie jedną metodę abstrakcyjną (może zawierać adnotację @FunctionalInterface). Wyrażenia lambda stanowią zwięzłą reprezentację instancji takiego interfejsu.

```
Java
@FunctionalInterface
interface StringTransformer {
    String transform(String input);
}
```

```

public class Main {
    public static void main(String[] args) {
        // Implementacja własnego interfejsu za pomocą lambdy
        StringTransformer upper = text -> text.toUpperCase();
        System.out.println(upper.transform("hello"));
    }
}

```

Szablon Optional

- źródło: [Baeldung - Guide to Java 8 Optional](#)
Optional<T> to obiektowy kontener, który może zawierać wartość nie-null lub być pusty. Służy do jawnego reprezentowania braku wartości w typach zwracanych przez metody, co pomaga zapobiegać błędom NullPointerException. Dostarcza metody takie jak map(), flatMap(), filter(), orElse() oraz orElseGet().

Wątki

Tworzenie wątku, porównanie Thread i Runnable oraz start() a run()

- źródła: [GeeksforGeeks - Runnable vs Thread | start\(\) vs run\(\)](#)
Wątek można utworzyć poprzez dziedziczenie po klasie Thread lub implementację interfejsu Runnable (zalecane, ponieważ klasa może implementować inne interfejsy i dziedziczyć po innej klasie).
- Wywołanie **run()** po prostu wykonuje kod w bieżącym wątku bez uruchamiania nowego.
- Wywołanie **start()** tworzy nowy wątek w systemie operacyjnym i przydziela mu osobną przestrzeń wykonania, po czym wewnętrznie wywołuje metodę run().

Synchronizacja wątków

- źródło: [GeeksforGeeks - Synchronization in Java](#)
Synchronizacja zapobiega wyścigom (*race conditions*) przy dostępie wielu wątków do współdzielonych zasobów. W Javie realizuje się ją za pomocą słowa kluczowego synchronized (stłosowanego na metodzie lub jako blok kodu z podaniem obiektu-blokady) oraz sekcji krytycznych bazujących na wewnętrznych monitorach (*intrinsic locks*). Synchronizacja gwarantuje powtarzalność i wyłączność dostępu do danych dla jednego wątku naraz.

Cykl życia wątku

- źródło: [Baeldung - Life Cycle of a Thread in Java](#)
Wątek w Javie przechodzi przez następujące stany (zdefiniowane w Thread.State):
1. **NEW**: Utworzony, ale jeszcze nieuruchomiony metodą start().
 2. **RUNNABLE**: Gotowy do wykonania lub w trakcie wykonywania przez procesor.
 3. **BLOCKED**: Oczekuje na zwolnienie blokady monitora (synchronized).
 4. **WAITING**: Oczekuje nieokreślony czas na działanie innego wątku (np. po wait() lub join()).
 5. **TIMED_WAITING**: Oczekuje przez określony czas (np. po Thread.sleep(ms) lub wait(timeout)).
 6. **TERMINATED**: Zakończył wykonywanie metody run().

Swing

Porównanie AWT, Swing i JavaFX

- źródło: [GeeksforGeeks - AWT vs Swing vs JavaFX](#)

Cecha	AWT	Swing	JavaFX
Komponenty	Ciężkie (<i>heavyweight</i> – zależne od OS)	Lekkie (<i>lightweight</i> – pisane w Javie)	Lekkie, nowoczesny render
Wygląd	Natywny dla systemu OS	Niezależny, konfigurowalny (Look & Feel)	Zaawansowany (CSS, FXML)
Wydajność	Niska elastyczność	Wysoka elastyczność, wolniejsza grafika	Bardzo wysoka (akceleracja GPU)

Zdarzenia i ich obsługa w Swingu

- źródło: [Oracle Java Tutorial - Writing Event Listeners](#)

Obsługa zdarzeń w Swingu opiera się na wzorcu *Observer* (Delegation Event Model). Komponenty graficzne (źródła zdarzeń, np. JButton) generują obiekty zdarzeń (np. *ActionEvent*) po wykonaniu akcji przez użytkownika. Aby zareagować, rejestruje się na komponencie słuchacza zdarzeń (np. *ActionListener*) implementującego odpowiedni interfejs. Zdarzenia GUI są zawsze przetwarzane w dedykowanym wątku – *Event Dispatch Thread* (EDT).